Análise de Cenários de Falha em Microsserviços: Uma Abordagem Baseada em Observabilidade

Lucas Alves Meneses[1], **Cidcley Teixeira de Souza**[2]

[1] Departamento de Telemática - DTEL
Instituto Federal do Ceará - IFCE
Ceará, Brasil

lucas.alves.meneses06@aluno.ifce.edu.br, cidcley@ifce.edu.br

Abstract. *This paper investigates the application of observability practices for detecting and mitigating failures in microservices systems. The research uses an environment developed in Java Spring Boot, integrating tools such as Prometheus, Grafana, Loki and Jaeger for monitoring, visualization, log aggregation and distributed tracing, respectively. Scenarios of high latency, cyclical dependency, resource exhaustion and communication failures were simulated, evidencing the ability of the tools to identify anomalies and diagnose problems. The results reinforce that the integration of advanced observability practices is essential to increase the resilience and reliability of distributed environments, contributing to the maintenance and evolution of complex systems.*

Resumo. *Este artigo investiga a aplicação de práticas de observabilidade para a detecção e mitigação de falhas em sistemas de microsserviços. A pesquisa utiliza um ambiente desenvolvido em Java Spring Boot, integrando ferramentas como Prometheus, Grafana, Loki e Jaeger para monitoramento, visualização, agregação de logs e rastreamento distribuído, respectivamente. Foram simulados cenários de latência elevada, dependência cíclica, esgotamento de recursos e falhas de comunicação, evidenciando a capacidade das ferramentas em identificar anomalias e diagnosticar problemas. Os resultados reforçam que a integração de práticas avançadas de observabilidade é essencial para aumentar a resiliência e a confiabilidade de ambientes distribuídos, contribuindo para a manutenção e evolução de sistemas complexos.*

1. Introdução

Nos últimos anos, a arquitetura de microsserviços consolidou-se como uma escolha predominante no desenvolvimento de sistemas modernos, impulsionada pela modularidade, escalabilidade e flexibilidade tecnológica que oferece [Newman 2015, Thönes 2015]. Essa abordagem permite o desenvolvimento de um único aplicativo como um conjunto de pequenos serviços, cada um executando seu próprio processo e se comunicando com mecanismos leves [Sousa 2024]. Empresas globais, como Netflix, Amazon e Uber, adotaram essa abordagem para lidar com a crescente complexidade e demandas de desempenho de seus ambientes distribuídos [Nunes 2019]. Essa evolução, no entanto, implica desafios consideráveis, sobretudo no diagnóstico e na resolução de problemas decorrentes da interação entre inúmeros serviços independentes, tornando mais complexo acompanhar e mapear o fluxo completo das requisições dentro do sistema

[Sousa 2024, Gomes et al. 2022]. Em uma arquitetura onde cada componente opera de forma autônoma, questões como timeouts, indisponibilidade de recursos e falhas na comunicação podem emergir de maneira inesperada, comprometendo não apenas a confiabilidade, mas também o desempenho global do sistema. Conforme destacado por [Newman 2015], a decomposição dos sistemas em microsserviços, embora traga benefícios evidentes, também exige uma abordagem diferenciada para o monitoramento e a manutenção de sua integridade, especialmente em sistemas cloud-native que são dinâmicos e difíceis de raciocinar [Majors et al. 2022, Nunes 2019, Poole et al. 2019].

Nesse cenário, a observabilidade se apresenta como uma solução estratégica e diferenciada em relação ao monitoramento tradicional, que geralmente se restringe a métricas predefinidas e foca em ”known-unknowns”(problemas conhecidos) [Majors et al. 2022]. A observabilidade é a capacidade de entender o que está acontecendo dentro de um sistema, medindo e analisando seus estados internos por meio de métricas, logs e rastreamentos [Sousa 2024, Nunes 2019]. Ao adotar práticas que envolvem a coleta integrada desses pilares (métricas, logs e rastreamento distribuído), os engenheiros ganham uma visão mais profunda do comportamento dos sistemas, permitindo a identificação e a resolução de falhas antes que estas atinjam níveis críticos [Gomes et al. 2022]. Essa abordagem dinâmica possibilita a exploração dos dados de forma contínua, favorecendo a detecção de anomalias e a investigação aprofundada das causas raízes dos problemas, focando nos ”unknown-unknowns”(problemas imprevisíveis) [Majors et al. 2022]. A padronização da instrumentação com tecnologias como OpenTelemetry [Boten 2022] é fundamental para essa abordagem coesa, unificando a coleta de telemetria e evitando o vendor lock-in [Parker et al. 2019]. Como ilustrado na Figura 1 – Arquitetura Típica de Microsserviços, a integração entre os diversos componentes e ferramentas de monitoramento permite a visualização clara das interações e a identificação de pontos vulneráveis em ambientes complexos [Parker et al. 2019].

Diante da crescente adoção dos microsserviços e dos desafios inerentes à sua manutenção, o problema central deste estudo reside na investigação de como a observabilidade pode ser aplicada para detectar, diagnosticar e mitigar diferentes cenários de falha em sistemas distribuídos.

Para responder a essa questão, o objetivo geral deste trabalho é analisar a eficácia das práticas de observabilidade na detecção e mitigação de falhas em sistemas de microsserviços [Sousa 2024, Gomes et al. 2022]. Assim, a pesquisa propõe-se a implementar um sistema de microsserviços onde cenários de falha são simulados, a configurar ferramentas especializadas para monitoramento e coleta de dados relevantes, e a analisar esses dados com o intuito de identificar a causa raiz dos problemas, propondo soluções que aprimorem a resiliência dos sistemas [Nunes 2019, Majors et al. 2022, Miles 2019]. Essa abordagem visa, de forma integrada, unir teoria e prática, contribuindo para a melhoria contínua da gestão de sistemas distribuídos e para o alcance de altos níveis de confiabilidade e resiliência, aspectos essenciais da engenharia de confiabilidade de sites (SRE) [Beyer et al. 2016, Beyer et al. 2018, Hidalgo 2020, Posta and Sutter 2018].

A motivação para a realização deste estudo decorre da constatação de que falhas em sistemas distribuídos podem acarretar prejuízos financeiros significativos e impactar negativamente a reputação das empresas [Majors et al. 2022]. Estudos indicam que o custo médio do downtime pode atingir milhares de dólares por minuto, depen-

dendo do setor, o que reforça a importância de estratégias proativas na identificação e resolução de problemas [Basili et al. 1994]. Em meio a esse cenário, a observabilidade revela-se como uma ferramenta essencial, capacitando as equipes técnicas a enfrentar a complexidade crescente dos sistemas distribuídos e a garantir que os serviços oferecidos atendam às expectativas dos usuários de maneira eficiente e resiliente [Sousa 2024, Gomes et al. 2022, Majors et al. 2022].

2. Fundamentação teórica

Para compreender como a observabilidade pode ser utilizada na detecção e mitigação de falhas em sistemas de microsserviços, é fundamental estabelecer uma base teórica sólida que aborde os principais conceitos relacionados ao tema. Este capítulo analisa os fundamentos da arquitetura de microsserviços, os conceitos e pilares da observabilidade e os principais cenários de falha em sistemas distribuídos, proporcionando uma visão integrada que sustenta as abordagens adotadas neste trabalho.

2.1. Arquitetura de Microsserviços

A arquitetura de microsserviços é um modelo de design de software que organiza sistemas como um conjunto de serviços independentes e modulares [Newman 2015, Thönes 2015]. Cada serviço é responsável por uma funcionalidade específica e pode ser desenvolvido, implantado e escalado de forma autônoma [Newman 2015]. Essa abordagem contrasta fortemente com a arquitetura monolítica, na qual todos os componentes estão integrados em uma única unidade, dificultando a manutenção e a escalabilidade em ambientes de alta complexidade [Nunes 2019, Soldani et al. 2018]. Entre as principais vantagens dos microsserviços, destacam-se a escalabilidade independente, a facilidade de manutenção e a flexibilidade tecnológica, que possibilita o uso de diferentes linguagens e frameworks para serviços distintos [Newman 2015, Nunes 2019, Poole et al. 2019]. Contudo, a independência dos componentes também acarreta desafios significativos, principalmente no que tange à comunicação entre serviços, à gestão de transações distribuídas e à identificação da causa raiz de falhas [Nunes 2019, Soldani et al. 2018, Majors et al. 2022]. Conforme salientado por [Newman 2015], a decomposição de um sistema em diversos serviços impõe uma complexidade adicional para a depuração, exigindo estratégias robustas para o diagnóstico e a resolução de problemas. A integração de ferramentas e componentes de observabilidade para proporcionar uma visão unificada e clara das interações em ambientes de microsserviços pode ser visualizada na Figura 1

2.2. Observabilidade em Sistemas Distribuídos

A observabilidade pode ser definida como a capacidade de inferir o estado interno de um sistema a partir de suas saídas externas [Majors et al. 2022, Nunes 2019]. Diferentemente do monitoramento tradicional — que se concentra na coleta e visualização de dados predefinidos, como métricas e logs de erros, lidando com ”known-unknowns”—, a observabilidade proporciona uma visão mais profunda e dinâmica, permitindo responder às perguntas “por que algo está errado?” e “como corrigir?” por meio da análise integrada de diversas fontes de informação e focando em ”unknown-unknowns”[Majors et al. 2022, Nunes 2019]. Essa abordagem é especialmente útil em sistemas distribuídos, onde uma única requisição pode transitar por múltiplos serviços, e a simples visualização de métricas pode não ser suficiente para identificar a origem de um problema [Majors et al. 2022].

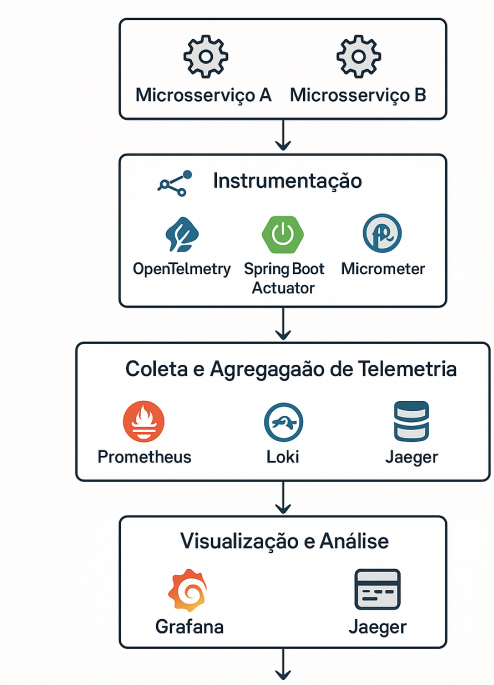


Figura 1. Cenário Implementado. Fonte: Próprio Autor

A observabilidade se sustenta em pilares fundamentais, embora o foco esteja na análise e nas práticas de depuração que esses dados permitem, e não apenas nos tipos de dados em si [Majors et al. 2022]. São eles:

1. **Logs:** São registros sequenciais de eventos que documentam ocorrências no sistema, oferecendo um contexto detalhado dos processos e auxiliando na identificação das causas dos problemas [Nunes 2019]. Enquanto as métricas indicam *o quê* pode estar errado, os logs frequentemente esclarecem *por que* uma falha ocorreu [Majors et al. 2022]. Ferramentas como o **Loki** facilitam a agregação, consulta e correlação desses registros, sendo essenciais para a análise forense de incidentes [Sousa 2024, Gomes et al. 2022]. A utilização de eventos estruturados é crucial para uma análise eficaz de logs [Majors et al. 2022].
2. **Métricas:** Representam dados numéricos que descrevem o desempenho e a saúde do sistema em tempo real. Indicadores como latência, taxa de erros, uso de CPU e tempo médio de resposta permitem identificar tendências e anomalias [Nunes 2019]. Ferramentas como o **Prometheus** são amplamente utilizadas para coletar, armazenar e analisar essas séries temporais, possibilitando a configuração de alertas para a detecção de comportamentos anômalos [Nunes 2019]. Contudo, [Majors et al. 2022] ressalta as limitações inerentes das métricas para depurar problemas inéditos, dada a sua natureza agregada. Essa distinção, que contrasta as limitações das métricas com o potencial dos eventos estruturados em logs para a depuração, é conceitualmente apresentada na Figura 2
3. **Rastreamento Distribuído:** Em ambientes compostos por microsserviços, o rastreamento distribuído mapeia a jornada de uma requisição através de múltiplos serviços, identificando gargalos e pontos de falha ao detalhar o tempo gasto em cada etapa do processo [Nunes 2019, Parker et al. 2019]. Soluções como o **Jae-**

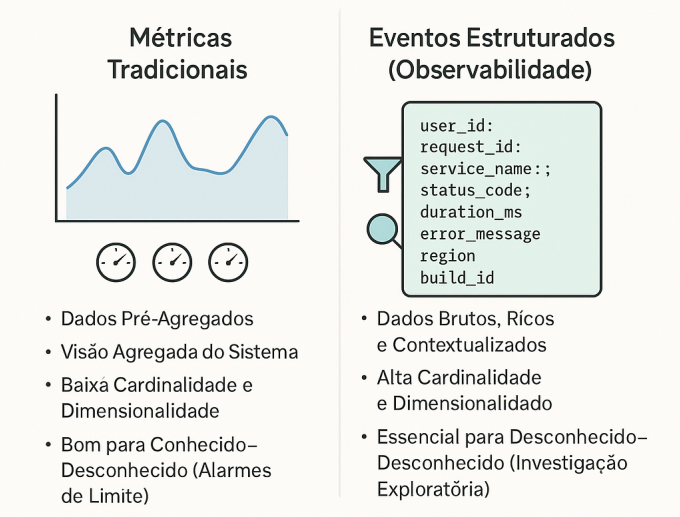


Figura 2. Comparativo Conceitual. Fonte: Próprio Autor

ger permitem visualizar essa cadeia de eventos, possibilitando a correlação entre diferentes pontos de falha e oferecendo insights precisos para a resolução de problemas [Sousa 2024, Gomes et al. 2022]. A capacidade de coletar **dados com alta cardinalidade e dimensionalidade** em traces é vital para depurar problemas inéditos [Majors et al. 2022]. A estrutura e o fluxo detalhado de um rastreamento distribuído podem ser melhor compreendidos ao observar a Figura 3, que ilustra visualmente a sequência de operações.

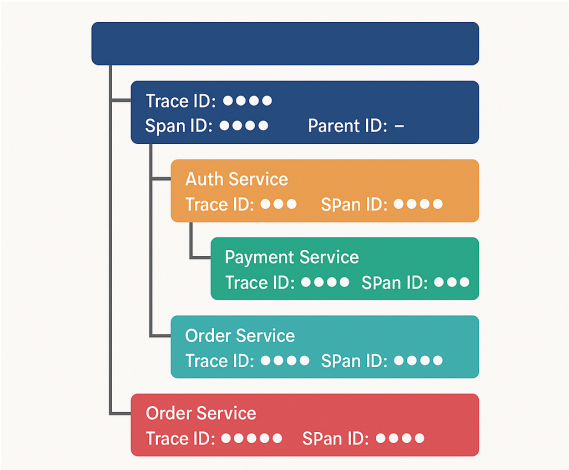


Figura 3. Trace Waterfall. Fonte: Próprio Autor

A integração dessas tecnologias é fundamental para uma abordagem coesa de observabilidade. O Spring Boot Actuator, por exemplo, atua como um facilitador, expondo automaticamente endpoints de saúde e métricas de desempenho. Através do Micrometer, essas métricas são padronizadas, tornando-as facilmente coletáveis por sistemas como o Prometheus [Sousa 2024, Gomes et al. 2022].

Complementarmente, o OpenTelemetry (OTel) foi empregado para garantir a instrumentação agnóstica de provedor, um padrão unificado para a geração de tele-

metria [Boten 2022, Majors et al. 2022]. Isso permite que um sistema produza de forma padronizada métricas, logs e, crucialmente, rastreamentos distribuídos (traces) [Sousa 2024, Gomes et al. 2022]. Essa padronização viabiliza que ferramentas como Loki (para logs) e Jaeger (para rastreamentos) possam consumir e correlacionar dados de diferentes microsserviços de forma eficiente [Sousa 2024, Gomes et al. 2022]. O resultado é uma visão unificada e contextualizada do comportamento do sistema em tempo real, independentemente da complexidade ou da novidade dos estados internos [Majors et al. 2022, Sousa 2024, Gomes et al. 2022].

A aplicação prática da observabilidade em sistemas de microsserviços é frequentemente complementada por abordagens como o Chaos Engineering, que envolve a injeção deliberada de falhas em ambientes controlados para testar a resiliência do sistema [Miles 2019]. Exemplos como o Chaos Monkey, popularizado pela Netflix, demonstram como a simulação de falhas aleatórias pode validar a eficácia dos mecanismos de redundância e dos processos de recuperação, evidenciando a sinergia entre a observabilidade e as práticas de resiliência [Miles 2019, Majors et al. 2022]. Essa sinergia entre observabilidade e práticas de resiliência, que é crucial para a gestão proativa de sistemas distribuídos e a manutenção de sua saúde e continuidade de serviço, pode ser ilustrada através de um ciclo contínuo de análise, como demonstrado na Figura 4.



Figura 4. O Ciclo de Análise da Observabilidade. Fonte: Próprio Autor

2.3. Cenários de Falha em Microsserviços

Em sistemas distribuídos baseados em microsserviços, as falhas podem ocorrer em diversas camadas e assumir diferentes formas, com impactos que variam conforme a natureza do problema. Entre os cenários mais comuns, destacam-se:

- **Falhas de Comunicação entre Serviços:** Essas falhas englobam timeouts, perda de pacotes e erros de rede, que podem comprometer a integridade das interações entre os serviços [Soldani et al. 2018, Nunes 2019]. Em ambientes de microsserviços, a comunicação via APIs remotas e a natureza distribuída aumentam a complexidade desses problemas [Soldani et al. 2018, Majors et al. 2022]. Um exemplo real ocorreu em 2021, quando problemas de comunicação entre

regiões na AWS (*Amazon Web Services*) afetaram aplicativos populares, evidenciando como a latência e a perda de pacotes podem desencadear comportamentos imprevisíveis.

- **Falhas de Componentes Individuais:** Quando componentes críticos, como bancos de dados ou sistemas de cache (*e.g.*, Redis), se tornam indisponíveis, o impacto pode se propagar por todo o sistema [Sousa 2024, Nunes 2019]. Uma falha no cache Redis, por exemplo, pode causar lentidão em sistemas de e-commerce durante picos de acesso, evidenciando a importância de estratégias de redundância [Sousa 2024].
- **Falhas de Recursos:** O esgotamento de recursos, como memória, CPU ou conexões, resulta geralmente de cargas inesperadas ou configurações inadequadas, afetando a capacidade do sistema de responder de forma eficiente [Gomes et al. 2022, Nunes 2019]. Um aumento súbito de usuários em um jogo online pode esgotar os recursos dos servidores, ocasionando interrupções temporárias [Gomes et al. 2022].
- **Falhas de Código:** Erros lógicos, exceções não tratadas e bugs podem emergir em condições específicas de produção, apesar de passarem despercebidos em testes convencionais [Majors et al. 2022]. Tais falhas podem, por exemplo, impedir a finalização de processos críticos em um sistema de delivery, afetando diretamente a experiência do usuário.
- **Falhas de Segurança:** Vulnerabilidades exploráveis podem comprometer a disponibilidade, integridade e confidencialidade dos sistemas [Nunes 2019]. Incidentes como o ataque DDoS que afetou o GitHub em 2018 demonstram como a sobrecarga de solicitações pode interromper serviços essenciais, reforçando a importância da observabilidade para a detecção precoce e mitigação desses ataques [Nunes 2019, Majors et al. 2022].

A identificação e mitigação desses cenários de falha são facilitadas pela observabilidade, que permite a correlação de dados provenientes de diferentes fontes e oferece uma visão abrangente do comportamento do sistema. Essa abordagem integrada é essencial para a construção de estratégias de resiliência que assegurem a continuidade dos serviços e a satisfação dos usuários, mesmo diante de condições adversas [Majors et al. 2022, Sousa 2024, Gomes et al. 2022, Nunes 2019].

2.4. Trabalhos Relacionados

Para contextualizar o presente estudo e identificar abordagens relevantes na literatura, esta seção apresenta uma revisão concisa de trabalhos relacionados, focando na observabilidade em microsserviços e sistemas distribuídos.

- **Sousa (2024)** focou na implementação e avaliação de práticas de observabilidade para microsserviços [Sousa 2024]. Utilizando um microsserviço em Spring Boot que se comunicava com Redis e PostgreSQL, o autor instrumentou a aplicação com Spring Boot Actuator e Micrometer, integrando Prometheus (métricas), Grafana (visualização), Loki (logs) e Jaeger com OpenTelemetry (rastreamento distribuído) [Sousa 2024]. O estudo simulou cenários de falha, como a interrupção do banco de dados, para demonstrar a capacidade das ferramentas em identificar e diagnosticar problemas como aumento na taxa de erros e latências. A visualização do service mesh com Kiali também foi explorada para identificar degradações

[Sousa 2024]. Em contraste, o presente estudo expande a análise para um conjunto mais abrangente de cenários de falha comuns em microsserviços, como dependência cíclica e esgotamento de recursos, e foca na demonstração prática da capacidade diagnóstica das ferramentas sem a exploração de service mesh para visualização específica.

- **Gomes et al. (2022)** investigaram a observabilidade de desempenho em arquiteturas monolíticas e de microsserviços, com destaque para aplicações *cloud-native* [Gomes et al. 2022]. Eles enfatizaram a observabilidade baseada em métricas, logs e traces, utilizando OpenTelemetry (OTel) como framework padronizado para geração de telemetria, integrado com Prometheus, Grafana, **Loki** e Jaeger [Gomes et al. 2022]. A pesquisa simulou diversos cenários de falha, como latência elevada, dependência cíclica, esgotamento de recursos e falhas de comunicação, comprovando a eficácia das ferramentas na identificação de anomalias e diagnóstico de problemas [Gomes et al. 2022]. Embora aborde cenários e ferramentas semelhantes, esta pesquisa se diferencia por sua concentração na aplicabilidade prática e na capacidade de diagnóstico detalhado dentro de um ambiente controlado baseado em Spring Boot, servindo como um guia para a identificação precisa de falhas específicas.
- **Nunes (2019)** buscou aprimorar a observabilidade do sistema SigSaúde, uma plataforma de prontuário eletrônico baseada em microsserviços, por meio da utilização de uma service mesh (Istio) [Nunes 2019]. A autora demonstrou que a service mesh poderia melhorar a visibilidade substancialmente sem exigir alterações no código dos microsserviços [Nunes 2019]. O trabalho abordou requisitos como tempo de execução e taxa de sucesso de requisições, rastreabilidade do fluxo de requisições, tamanho das requisições, e tratamento centralizado de logs para depuração [Nunes 2019]. Ferramentas como Prometheus, Grafana, Jaeger e Kiali foram utilizadas para visualização e exploração do estado da service mesh [Nunes 2019]. Embora alertas avançados não fossem um recurso nativo do Istio na versão estudada, a capacidade de sinalização visual de degradação e falha foi observada [Nunes 2019]. Diferentemente do trabalho de Nunes, que foca na injeção de observabilidade via service mesh sem modificações no código, o presente estudo demonstra a eficácia da instrumentação direta das aplicações Spring Boot com OpenTelemetry para a coleta e correlação de telemetria, enfatizando a sinergia entre as ferramentas de observabilidade para um diagnóstico preciso de falhas.

Em resumo, os trabalhos analisados convergem na validação da importância dos pilares da observabilidade (métricas, logs e traces) e na eficácia de ferramentas como Prometheus, Grafana, Loki e **Jaeger** em ambientes de microsserviços [Sousa 2024, Gomes et al. 2022, Nunes 2019]. Enquanto [Sousa 2024] e [Gomes et al. 2022] focam na instrumentação direta das aplicações com OpenTelemetry e na simulação de falhas para diagnóstico, [Nunes 2019] apresenta a alternativa da service mesh para injetar observabilidade de forma transparente, sem a necessidade de modificação do código da aplicação. Essa sinergia nas abordagens e ferramentas valida as escolhas metodológicas adotadas no presente artigo para a análise de cenários de falha em microsserviços [Majors et al. 2022].

3. Metodologia

Para comprovar a eficácia da observabilidade na identificação e diagnóstico de problemas comuns em sistemas de microsserviços, foi implementado um ambiente experimental controlado baseado em Java Spring Boot [Sousa 2024, Gomes et al. 2022]. Este ambiente foi desenvolvido utilizando Java Spring Boot, uma escolha fundamentada em sua ampla popularidade no desenvolvimento de microsserviços em Java e em sua integração nativa com bibliotecas que facilitam a coleta de métricas, logs e rastreamentos distribuídos [Sousa 2024, Gomes et al. 2022].

O ambiente experimental consistiu em dois microsserviços interconectados [Sousa 2024, Gomes et al. 2022], desenhados para simular um sistema distribuído básico e permitir a observação das interações e propagação de falhas.

Nesse ambiente, foram cuidadosamente integradas e configuradas as seguintes ferramentas de observabilidade, que constituem os pilares da abordagem adotada [Sousa 2024, Gomes et al. 2022]:

- **Prometheus**: Esta ferramenta foi a responsável pela coleta e armazenamento de métricas em séries temporais, por meio de sua integração com o Micrometer [Sousa 2024, Gomes et al. 2022]. As métricas representam dados numéricos que descrevem o desempenho e a saúde do sistema em tempo real [Majors et al. 2022, Nunes 2019]. O Prometheus foi configurado para identificar tendências e anomalias nos indicadores em tempo real, sendo fundamental para a detecção de comportamentos inesperados [Nunes 2019].
- **Grafana**: Utilizado para a criação de dashboards dinâmicos que permitiram a visualização em tempo real das métricas coletadas pelo Prometheus [Sousa 2024, Gomes et al. 2022]. Por meio de gráficos de séries temporais e outros recursos visuais, o Grafana possibilitou a identificação rápida de anomalias e a compreensão intuitiva da degradação do desempenho do sistema [Sousa 2024, Gomes et al. 2022].
- **Loki**: Empregado para a agregação e consulta de logs gerados pelos microsserviços [Sousa 2024, Gomes et al. 2022]. O Loki foi essencial para o rastreamento detalhado de eventos e erros [Sousa 2024, Gomes et al. 2022], oferecendo um contexto minucioso sobre os processos e, crucialmente, ajudando a esclarecer ”por que”uma falha ocorreu, complementando as métricas que indicam apenas ”o quê”[Majors et al. 2022]. A utilização de eventos estruturados nos logs foi primordial para uma análise forense eficaz de incidentes [Majors et al. 2022].
- **Jaeger**: Implementado para o rastreamento distribuído, o Jaeger permitiu mapear a jornada completa de uma requisição à medida que ela transitava entre os microsserviços [Sousa 2024, Gomes et al. 2022, Parker et al. 2019, Nunes 2019]. Ele foi fundamental para identificar gargalos e pontos de falha na comunicação, detalhando o tempo gasto em cada etapa do processo e possibilitando a correlação de diferentes pontos de falha [Sousa 2024, Gomes et al. 2022, Parker et al. 2019, Nunes 2019].
- **OpenTelemetry (OTel)**: Uma tecnologia chave empregada para garantir a instrumentação agnóstica de provedor [Majors et al. 2022, Boten 2022]. O OTel é um padrão unificado para a geração de telemetria, incluindo métricas, logs e, principalmente, rastreamentos distribuídos (traces) [Boten 2022]. Neste ambiente, ele foi empregado para os rastreamentos distribuídos e os logs, enviados

por um OpenTelemetry Collector ao Jaeger e ao Loki; as métricas foram expostas pelo Micrometer e coletadas pelo Prometheus. Sua utilização foi crucial para evitar o ”vendor lock-in”[Majors et al. 2022, Boten 2022] e para viabilizar que ferramentas como Loki e Jaeger pudessem consumir e correlacionar dados de diferentes microsserviços de forma eficiente e padronizada [Majors et al. 2022]. Isso resultou em uma visão unificada e contextualizada do comportamento do sistema em tempo real, independentemente da sua complexidade ou de estados internos inéditos [Majors et al. 2022]. O OpenTelemetry também se integra com bibliotecas como o Spring Boot Actuator e o Micrometer para facilitar a coleta e correlação de dados [Sousa 2024, Gomes et al. 2022].

A aplicação prática da observabilidade é frequentemente complementada por abordagens como o Chaos Engineering, que envolve a injeção deliberada de falhas em ambientes controlados para testar a resiliência do sistema [Miles 2019]. Exemplos como o Chaos Monkey, popularizado pela Netflix, demonstram como a simulação de falhas aleatórias pode validar a eficácia dos mecanismos de redundância e dos processos de recuperação, evidenciando a sinergia entre a observabilidade e as práticas de resiliência [Miles 2019]. Embora não seja o foco principal da metodologia de simulação de falhas específica deste estudo, essa sinergia entre observabilidade e práticas de resiliência reforça a relevância da abordagem adotada [Miles 2019].

Para validar a capacidade de detecção e diagnóstico das ferramentas de observabilidade, foram implementadas simulações de diferentes cenários de falha, seguindo abordagens similares a estudos prévios na área [Sousa 2024, Gomes et al. 2022]:

1. **Latência Elevada:**
    - **Configuração**: O Microsserviço A, simulando uma funcionalidade como processamento de pedidos, realizou uma chamada HTTP para o Microsserviço B (e.g., serviço de pagamentos). No Microsserviço B, foi introduzido intencionalmente um delay artificial (um atraso) antes de processar a requisição, simulando um processamento demorado ou uma dependência externa lenta [Sousa 2024, Gomes et al. 2022].
    - **Observação pelas Ferramentas**: Essa configuração permitiu que o Prometheus coletasse métricas de tempo de resposta que demonstrariam o aumento da latência. O Jaeger registraria o tempo de processamento específico no serviço afetado, identificando o ponto exato do atraso. O Grafana, através de gráficos de séries temporais, apresentaria visualmente a degradação do desempenho, evidenciando o aumento da latência de forma clara [Sousa 2024, Gomes et al. 2022]. Essa configuração e a visibilidade proporcionada pelas ferramentas são conceitualmente representadas no Diagrama 5.
2. **Dependência Cíclica:**
    - **Configuração**: Nesta simulação, o Microsserviço A foi configurado para chamar o Microsserviço B, que, por sua vez, realizou uma nova chamada de volta ao Microsserviço A. Um limite de chamadas foi configurado para controlar o ciclo e evitar loops infinitos, permitindo uma observação controlada [Sousa 2024, Gomes et al. 2022].
    - **Observação pelas Ferramentas**: O rastreamento distribuído do Jaeger seria crucial para capturar essa cadeia de chamadas cíclicas, visualizando

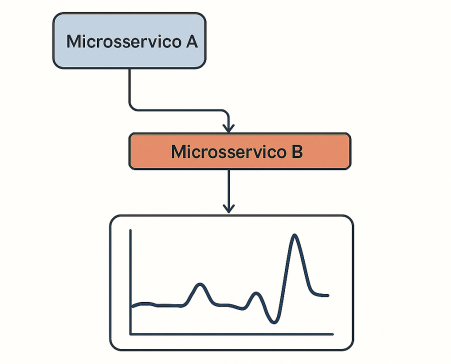


Figura 5. Diagrama Latência Elevada. Fonte: Próprio Autor

o loop. Paralelamente, os logs centralizados no Loki registrariam erros como timeouts ou exaustão de retentativas, fornecendo o diagnóstico da falha na comunicação. O Prometheus seria configurado para monitorar um aumento anormal nas requisições, indicando um problema de tráfego relacionado ao ciclo [Sousa 2024, Gomes et al. 2022]. Esse processo de identificação e diagnóstico de problemas complexos, que envolve a correlação de dados e a análise contínua do sistema, pode ser visualizado na Figura 6.

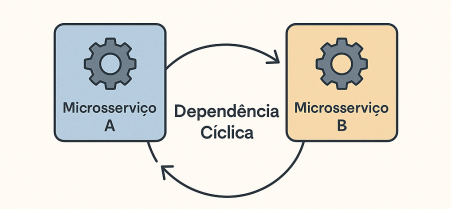


Figura 6. Diagrama Dependência Cíclica. Fonte: Próprio Autor

3. **Esgotamento de Recursos:**
   - **Configuração**: Um endpoint específico foi implementado no Microsserviço A para simular o esgotamento de recursos através da execução de operações intensivas que consumiam grandes quantidades de memória e CPU, como loops computacionais pesados ou alocação excessiva de memória [Sousa 2024, Gomes et al. 2022].
   - **Observação pelas Ferramentas**: O Prometheus monitoraria os indicadores de uso de CPU e memória, evidenciando o aumento progressivo desses recursos. O Grafana exibiria gráficos que ilustrariam o consumo e a tendência de esgotamento. O Loki registraria eventos críticos, como `OutOfMemoryError` ou reinicializações de container, confirmando o impacto da simulação na estabilidade do serviço [Sousa 2024, Gomes et al. 2022]. Esse processo de detecção e diagnóstico de anomalias por meio da integração de métricas e logs é um componente chave, que é um conceito fundamental para a gestão proativa de sistemas distribuídos, conforme apresentado na Figura 7
4. **Falhas de Comunicação em Rede:**

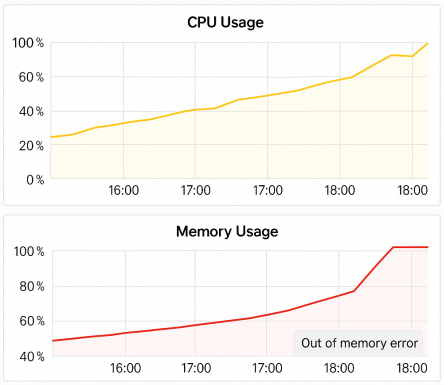


Figura 7. Diagrama Esgotamento de Recursos. Fonte: Próprio Autor

- **Configuração**: Para simular problemas de comunicação entre os microsserviços, foram aplicadas duas abordagens: a utilização da ferramenta de rede `tc` (Traffic Control) no ambiente Linux para introduzir atrasos ou perda de pacotes, e, alternativamente, a interrupção temporária do Microsserviço B enquanto o Microsserviço A tentava se comunicar com ele [Sousa 2024, Gomes et al. 2022].
- **Observação pelas Ferramentas**: Essas condições permitiriam que o Prometheus captasse métricas que indicassem um aumento significativo na taxa de erros (por exemplo, códigos HTTP 5xx). O Jaeger destacaria as falhas nas chamadas inter-serviços, evidenciando a interrupção do fluxo. O Loki registraria erros específicos, como `ConnectionRefused` ou `GatewayTimeout`, complementando a análise da causa raiz da falha de comunicação [Sousa 2024, Gomes et al. 2022]. Sendo brevemente ilustrado na Figura 8.

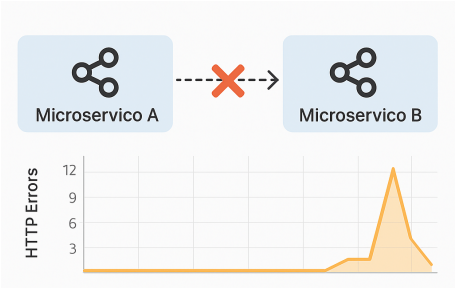


Figura 8. Diagrama Falhas de Comunicação em Rede. Fonte: Próprio Autor

Cada uma dessas simulações foi meticulosamente implementada para que as ferramentas de observabilidade pudessem desempenhar seus papéis específicos e complementares: o Prometheus para a coleta de métricas, o Grafana para a visualização dos dados, o Loki para a centralização dos logs e o Jaeger para o rastreamento distribuído [Sousa 2024, Gomes et al. 2022]. Essa abordagem metodológica possibilitou a criação de um ambiente robusto e controlado, onde a interação entre os microsserviços e as ferramentas de observabilidade pode ser explorada de forma sistemática [Sousa 2024, Gomes et al. 2022].

É importante ressaltar que a implementação deste ambiente e as simulações não visaram apresentar resultados estatísticos ou comparativos [Sousa 2024, Gomes et al. 2022]. O objetivo primordial desta metodologia foi demonstrar a aplicabilidade prática das ferramentas de observabilidade para a identificação e diagnóstico de falhas em sistemas distribuídos [Sousa 2024, Gomes et al. 2022]. Essa abordagem metodológica serve como um guia prático para a replicação em ambientes reais, evidenciando como a integração entre Spring Boot, Prometheus, Grafana, Loki e Jaeger pode proporcionar uma solução eficiente para monitorar, diagnosticar e, futuramente, mitigar falhas em microsserviços [Sousa 2024, Gomes et al. 2022].

4. Resultados

A aplicação da metodologia proposta permitiu a obtenção de evidências claras sobre a eficácia das ferramentas de observabilidade integradas ao ambiente de microsserviços. Em todos os cenários simulados, os dados coletados por Prometheus, Grafana, Loki e Jaeger confirmaram o comportamento esperado do sistema, demonstrando sua capacidade de identificação e diagnóstico de falhas.

4.1. Cenário de Latência Elevada

No cenário de latência elevada, a introdução intencional de um delay artificial no processamento do Microsserviço B resultou em um aumento perceptível no tempo de resposta.

- O Prometheus capturou esse aumento de latência através de métricas de tempo de resposta.
- Os dashboards do Grafana evidenciaram a degradação do desempenho por meio de gráficos de séries temporais durante o período da simulação.
- Os traçados distribuídos do Jaeger foram cruciais para identificar o ponto exato onde o atraso foi introduzido (o serviço B), corroborando a eficiência da ferramenta na visualização dos fluxos de requisições.

Essa visibilidade, fornecida pelo Jaeger, Grafana e Prometheus, permite que as equipes foquem imediatamente as ações de mitigação, como otimizar o código do Microsserviço B, ajustar configurações de pool de conexões ou escalar recursos horizontalmente para restaurar rapidamente o desempenho. A diferença de latência e a degradação do desempenho podem ser visualmente observadas através dos rastreamentos distribuídos do Jaeger, como ilustrado nas **Figuras 9 e 10**, e nos gráficos de duração de requisição do Grafana, conforme a **Figura 11**.

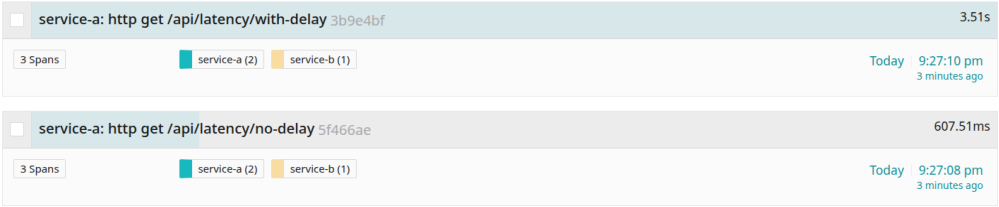


Figura 9. Jaeger, comparação com e sem delay. Fonte: Próprio Autor

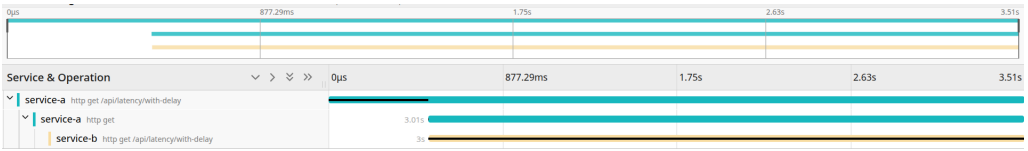


Figura 10. Jaeger, delay maior no serviço B. Fonte: Próprio Autor

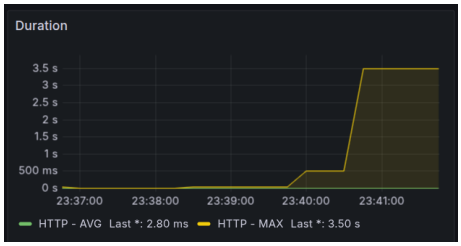


Figura 11. Grafana, grafico duração da requisição. Fonte: Próprio Autor

4.2. Cenário de Dependência Cíclica

A simulação de dependência cíclica revelou a ocorrência de um ciclo controlado de chamadas entre os microsserviços (Microsserviço A chamando B, que chamou A novamente).

- O rastreamento distribuído do Jaeger foi fundamental para evidenciar o loop de requisições.
- Os logs centralizados no Loki registraram cada salto do ciclo e o evento de limite atingido, todos com o mesmo identificador de rastreamento, confirmando a presença da dependência cíclica e demonstrando como a observabilidade pode apontar a origem de problemas complexos em sistemas interconectados.

A detecção e mapeamento do ciclo pelo Jaeger, que foi fundamental para evidenciar o loop de requisições conforme ilustrado na **Figura 12** (tracer waterfall) e na **Figura 13** (estatística de cada serviço), junto aos registros de cada salto e do limite atingido no Loki, oferecem a inteligência necessária para mitigar a falha por meio de intervenções arquitetônicas, como a reestruturação dos serviços, implementação de Circuit Breakers ou refinamento de padrões de comunicação assíncrona.

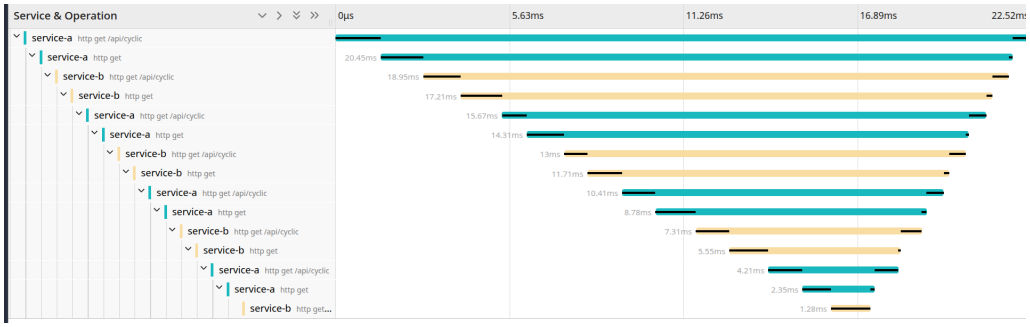


Figura 12. Jaeger, tracer waterfall. Fonte: Próprio Autor

4.3. Cenário de Esgotamento de Recursos

Na simulação de esgotamento de recursos, a execução de operações intensivas no Microsserviço A resultou em um consumo elevado de CPU e memória.

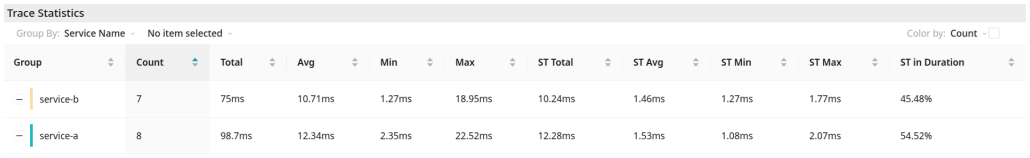


Figura 13. Jaeger, estatistica de cada serviço. Fonte: Próprio Autor

- As métricas coletadas pelo Prometheus evidenciaram o aumento progressivo desses recursos.
- Os dashboards do Grafana ilustraram essa tendência de forma clara.
- Adicionalmente, o Loki registrou mensagens de erro, como `OutOfMemoryError`, que confirmaram o impacto da simulação sobre a estabilidade do serviço.

Com esses dados do Prometheus, Grafana e Loki, o sistema de observabilidade forneceu as informações para mitigar o esgotamento de recursos. A visualização do uso de CPU pelo Grafana, como demonstrado na **Figura 14**, complementa as métricas e logs, fornecendo clareza sobre a tendência de consumo. As ações podem incluir otimização de algoritmos, implementação de limites de recursos mais rígidos em orquestradores (como Kubernetes) ou ajuste dinâmico de políticas de autoscaling.

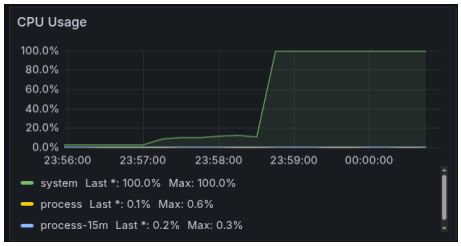


Figura 14. Grafana, uso de CPU. Fonte: Próprio Autor

4.4. Cenário de Falhas de Comunicação em Rede

A simulação de falhas de comunicação em rede foi validada pela introdução de atrasos e perda de pacotes (usando ‘tc‘ no Linux) e pela desativação temporária do Microsserviço B. Essa condição provocou problemas na comunicação entre os serviços.

- Um aumento nos códigos de erro HTTP foi capturado pelo Prometheus.
- Os rastreamentos do Jaeger evidenciaram a interrupção no fluxo de requisições entre os serviços.
- Os logs registrados pelo Loki complementaram a análise, destacando erros de tempo esgotado (`GatewayTimeout`) sob atraso e perda de pacotes e de host inalcançável (`UnknownHostException`) quando o Microsserviço B foi desligado.

O diagnóstico claro fornecido por Prometheus, Jaeger e Loki permite a mitigação de falhas de comunicação. A interrupção no fluxo de requisições evidenciada pelo Jaeger, conforme a **Figura 16**, e o aumento na contagem de erros no Grafana, ilustrado na **Figura 15**, fornecem insights cruciais. As ações podem incluir a configuração de políticas de retry e timeout mais adaptativas, implementação de health checks robustos ou investigação e correção de problemas na infraestrutura de rede subjacente, aumentando a resiliência da comunicação distribuída.



Figura 15. Grafana, contagem de erro. Fonte: Próprio Autor

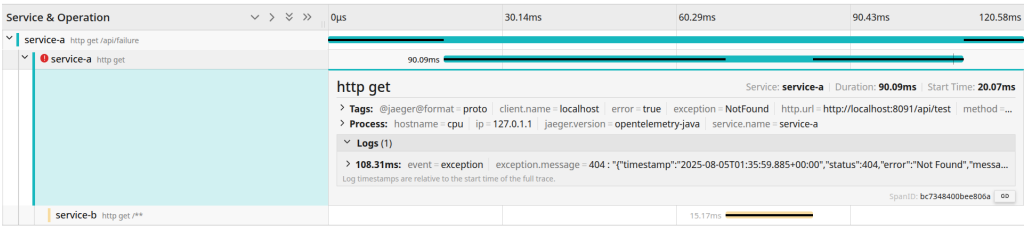


Figura 16. Jaeger, erro no serviço A. Fonte: Próprio Autor

4.5. Síntese dos Resultados

Em síntese, as evidências obtidas em cada cenário demonstram que a integração entre Spring Boot, Prometheus, Grafana, Loki e Jaeger oferece uma solução robusta para o monitoramento e diagnóstico de falhas em ambientes de microsserviços. As ferramentas de observabilidade não apenas capturaram os comportamentos anômalos esperados, mas também possibilitaram a identificação precisa dos pontos de falha, evidenciando o potencial desta abordagem para a implementação de sistemas resilientes e eficientes.

Cada ferramenta desempenhou um papel específico e complementar :

- O **Prometheus** foi eficaz na coleta de métricas e na identificação de variações nos indicadores de performance.
- O **Grafana** possibilitou a visualização intuitiva dos dados em tempo real.
- O **Loki** centralizou os registros de eventos críticos.
- O **Jaeger** foi fundamental para mapear o fluxo de requisições entre os microsserviços, revelando gargalos e falhas de comunicação.

Essa sinergia entre as tecnologias reforça a premissa de que sistemas distribuídos podem ser gerenciados de maneira proativa por meio de estratégias de observabilidade bem implementadas Os resultados obtidos validam a hipótese de que uma arquitetura de microsserviços, aliada a práticas avançadas de monitoramento, pode alcançar altos níveis de resiliência e confiabilidade, aspectos essenciais para a continuidade dos serviços em ambientes com alta complexidade operacional. As imagens coletadas, embora não apresentadas aqui, reforçam visualmente os dados e servem de guia para a replicação dos procedimentos em ambientes reais.

5. Conclusão

A realização deste estudo evidenciou, de forma clara, a importância da observabilidade na identificação e mitigação de falhas em ambientes de microsserviços. Ao integrar ferramentas modernas como Prometheus, Grafana, Loki e Jaeger a um ambiente desenvolvido

com Spring Boot, foi possível demonstrar como a coleta e a análise de métricas, logs e rastreamentos distribuídos fornecem uma visão abrangente e detalhada do comportamento dos sistemas. Os cenários de simulação – que abrangeram desde a introdução de latência artificial até a configuração intencionalmente incorreta de parâmetros críticos – confirmaram que a observabilidade não só detecta anomalias, mas também permite o diagnóstico preciso da origem dos problemas, facilitando a tomada de decisões e a implementação de estratégias corretivas.

Ao longo do desenvolvimento do projeto, observou-se que cada ferramenta desempenha um papel específico e complementar: o Prometheus se mostrou eficaz na coleta de métricas e na identificação de variações nos indicadores de performance, o Grafana possibilitou a visualização intuitiva dos dados em tempo real, o Loki centralizou os registros de eventos críticos, e o Jaeger foi fundamental para mapear o fluxo de requisições entre os microsserviços, revelando os gargalos e as falhas de comunicação. Essa sinergia entre as tecnologias evidencia a robustez da abordagem adotada e reforça a premissa de que sistemas distribuídos podem ser gerenciados de maneira proativa por meio de estratégias de observabilidade bem implementadas.

Os resultados obtidos validam a hipótese de que uma arquitetura de microsserviços, aliada a práticas avançadas de monitoramento, pode alcançar altos níveis de resiliência e confiabilidade, aspectos essenciais para a continuidade dos serviços em ambientes com alta complexidade operacional. Apesar de a presente pesquisa ter se concentrado na simulação de cenários controlados, os insights derivados deste trabalho oferecem subsídios importantes para a aplicação dessas técnicas em ambientes de produção, onde as implicações financeiras e operacionais de falhas podem ser significativas.

Em síntese, este estudo reforça a necessidade de uma abordagem integrada e sistemática para a gestão de microsserviços, na qual a observabilidade é tratada como um componente central para a manutenção da saúde do sistema. Trabalhos futuros podem expandir essa pesquisa, aplicando os métodos e ferramentas em cenários reais de produção e explorando a incorporação de novas tecnologias e práticas, como o uso de inteligência artificial para análise preditiva e a automação de respostas a incidentes. Essa evolução não só contribuirá para o aprimoramento contínuo dos sistemas distribuídos, mas também para o desenvolvimento de soluções mais inteligentes e resilientes, capazes de atender às demandas crescentes de desempenho e confiabilidade em ambientes cada vez mais complexos.

Portanto, a integração entre observabilidade e arquitetura de microsserviços não apenas representa uma tendência emergente no desenvolvimento de sistemas modernos, mas também se configura como uma estratégia indispensável para garantir a robustez e a eficiência de serviços críticos, impactando positivamente tanto a experiência dos usuários quanto a sustentabilidade operacional das organizações.

Referências

Basili, V. R., Caldiera, G., and Rombach, H. D. (1994). The goal question metric approach. In *Encyclopedia of Software Engineering*, pages 528–532. Wiley.

Beyer, B. et al. (2016). *Site Reliability Engineering*. O'Reilly Media, Inc.

Beyer, B. et al. (2018). *The Site Reliability Workbook*. O'Reilly Media, Inc.

Boten, A. (2022). *Cloud Native Observability with OpenTelemetry*. Packt Publishing.

Gomes, F., Mores, V., Rocha, L., Rego, P., and Trinta, F. (2022). Observabilidade de desempenho de arquiteturas monolíticas e microsserviços com opentelemetry. *CREAT – Universidade Federal do Ceará – Fortaleza – CE – Brasil*.

Hidalgo, A. (2020). *Implementing Service Level Objectives*. O'Reilly Media, Inc.

Majors, C., Fong-Jones, L., and Miranda, G. (2022). *Observability Engineering: Achieving Production Excellence*. O'Reilly Media, Inc.

Miles, R. (2019). *Chaos Engineering Observability*. O'Reilly.

Newman, S. (2015). *Building Microservices: Designing Fine-Grained Systems*. O'Reilly Media, Inc.

Nunes, J. S. (2019). Implantando a observabilidade do sistema sigsaúde através da utilização de service mesh. Master's thesis, Universidade Federal do Rio Grande do Norte, Natal, RN.

Parker, A. et al. (2019). *Distributed Tracing in Practice*. O'Reilly Media, Inc.

Poole, S. et al. (2019). *Developing Open Cloud Native Microservices*. O'Reilly Media, Inc.

Posta, C. and Sutter, B. (2018). *Introducing Istio Service Mesh for Microservices: Build and Deploy Resilient, Fault-Tolerant Cloud-Native Applications*. O'Reilly Media, Inc.

Soldani, J., Tamburri, D. A., and Heuvel, W.-J. V. D. (2018). The pains and gains of microservices: A systematic grey literature review. *Journal of Systems and Software*, 146:215–232.

Sousa, W. W. A. d. (2024). *ANÁLISE E IMPLEMENTAÇÃO DE PRÁTICAS DE OBSERVABILIDADE ESPECÍFICA PARA MICROSSERVIÇOS*. Trabalho de conclusão de curso, Universidade Federal do Ceará, Sobral.

Thönes, J. (2015). Microservices. *IEEE software*, 32(1):116–116.